DATA 260 Homework 8

Priyank Mehta | SJSU ID 019152562

GitHub repository: https://github.com/priyankmehta5/data260-2562

Personal configuration

Value	Configuration
SID4	2562
PORT_BASE	8762 = 8000 + (2562 mod 900)
PREFIX	s2562
SEED	2562
VERIFY_SEED	262562
DOMAIN_ID	2
Domain	Municipal transit incidents
Processor	Intel Core i7-8750H CPU @ 2.20 GHz
Memory and operating system	15.86 GB RAM; Windows 10
Graphics	NVIDIA GeForce GTX 1050 Ti; Intel UHD Graphics 630
Local model	qwen3:1.7b
Verification commit	2ba9aaa6a6c0e148daf43499b9b12decebc65e72

The incident service uses Kafka for CREATE_INCIDENT and GET_INCIDENT requests. Planner, Writer, and Reviewer run in separate processes and exchange messages through Kafka topics. The live run passed all 13 verification checks, including the four failure experiments; the offline suite passed 14 tests.

The HW8 agent stages use deterministic transit-domain rules. The local model above records the project configuration; no model generation or token measurements are used in these messaging experiments.

Part 1 Kafka request and response

Service deployment

Kafka, the incident worker, FastAPI, Planner, Writer, and Reviewer run as separate Docker services. FastAPI is published at http://127.0.0.1:8762. The API wrapper sends messages to Kafka and waits for a matching reply; incident creation and retrieval execute inside the worker.

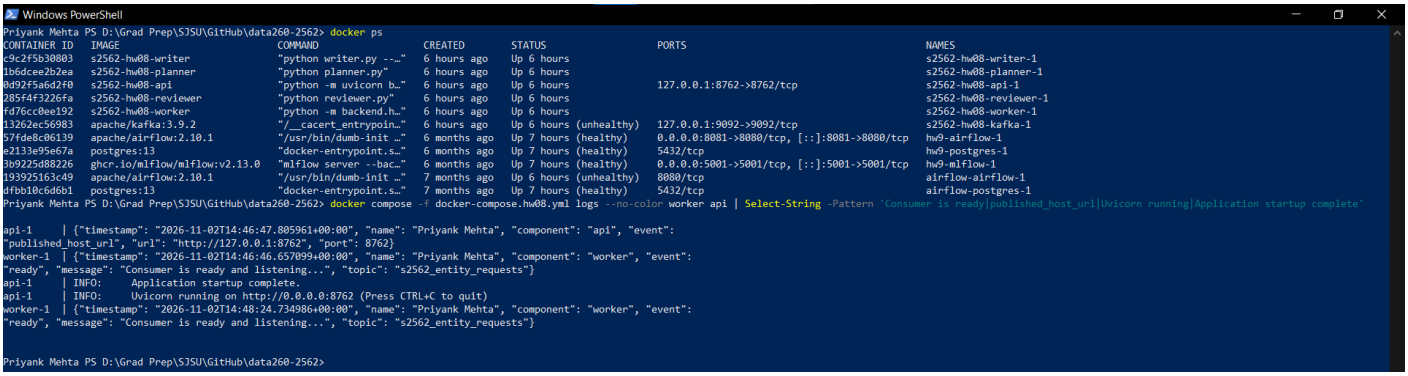


Figure 1. Part 1 deployment and worker/API startup logs.

Request and response topics

Topic	Purpose
s2562_entity_requests	Client/API requests to the incident worker
s2562_entity_responses	Worker replies matched by correlation_id

Request message schema

Field	Type and rule
correlation_id	Nonempty string generated by the caller; preserved unchanged
reply_to	s2562_entity_responses
operation	CREATE_INCIDENT or GET_INCIDENT
data	CREATE: title, description, incident_code, severity, agency_id. GET: incident_id (UUID).

Part 1 Operations and validation

Required fields are validated before inserting an incident into the worker's Python dictionary. Each successful creation receives a UUID. GET validates the UUID and returns the stored fields. Unknown operations, missing fields, and nonexistent incidents return success=false with the original correlation_id.

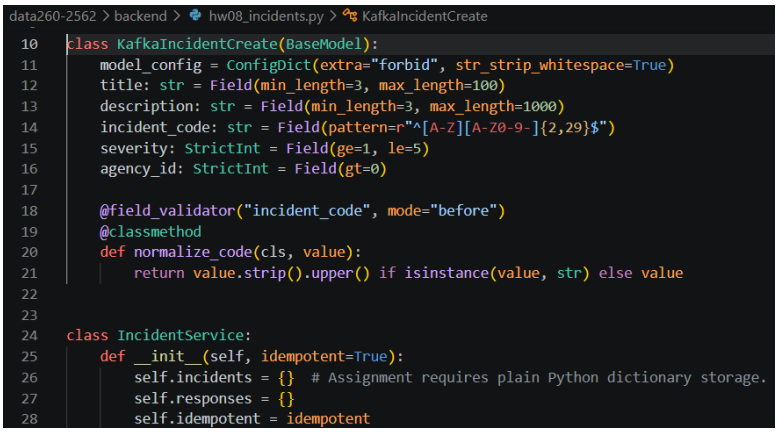


Figure 2. Part 1 incident fields, validation constraints, and dictionary storage (backend/hw08_incidents.py).

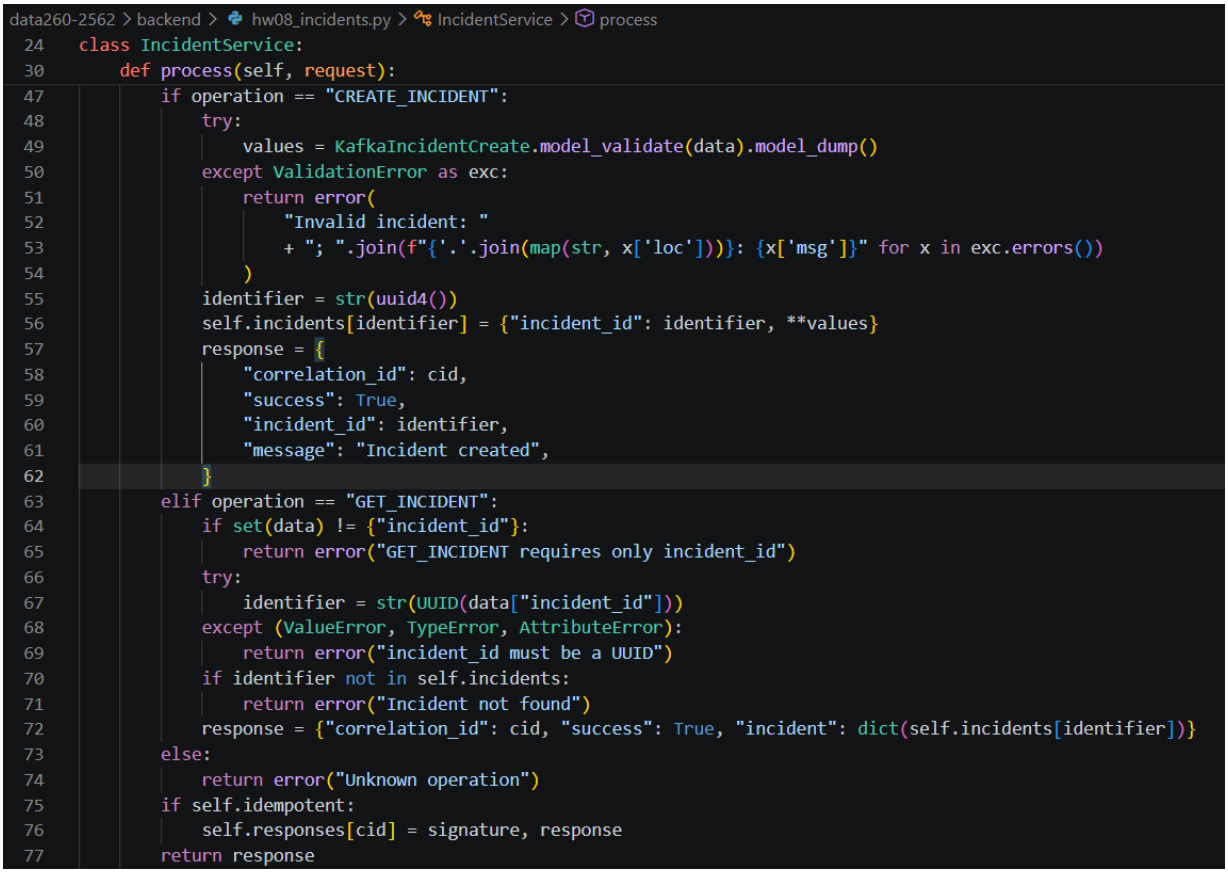


Figure 3. Part 1 UUID creation, retrieval, response envelopes, and error handling.

Part 1 CREATE then GET demonstration

homework_demo.py creates the request correlation ID before invoking FastAPI. It extracts the generated incident_id from CREATE and uses it in GET. Each operation has its own correlation ID, and the helper asserts that its response echoes that ID.

```python
data260-2562 > homework_demo.py > ...
14  async def demonstrate(via_api=True):
28          async def call(request):
29              emit("demo", "http_request" if via_api else "kafka_request", message=request)
30              if rpc:
31                  response = await rpc.request(request)
32              else:
33                  result = await client.post("http://127.0.0.1:8762/api/hw08/incidents", json=request)
34                  result.raise_for_status()
35                  response = result.json()
36              assert response["correlation_id"] == request["correlation_id"], "Correlation changed in transit"
37              emit("demo", "response", message=response)
38              return response
39
40          try:                                          (variable) values: dict[str, Any]
41              create = make_request("CREATE_INCIDENT", values)  # Generated once here; API and worker preserve it.
42              created = await call(create)
43              assert created["success"], created
44              get = make_request("GET_INCIDENT", {"incident_id": created["incident_id"]})
45              retrieved = await call(get)
46              assert retrieved["success"] and retrieved["incident"] == {"incident_id": created["incident_id"], **values}
47              output = {
48                  "create_request": create,
49                  "create_response": created,
50                  "get_request": get,
51                  "get_response": retrieved,
52                  "correlation_preserved": True,
53                  "via_api": via_api,
54              }
55              RAW.mkdir(parents=True, exist_ok=True)
56              (RAW / "rpc_demo.json").write_text(json.dumps(output, indent=2), encoding="utf-8")
57              print("Priyank Mehta - PASS CREATE_INCIDENT -> GET_INCIDENT; both correlation IDs preserved", flush=True)
58              return output
59          finally:
60              if rpc:
61                  await rpc.stop()
```

Figure 4. Part 1 demonstration sequence and correlation assertions (homework_demo.py).

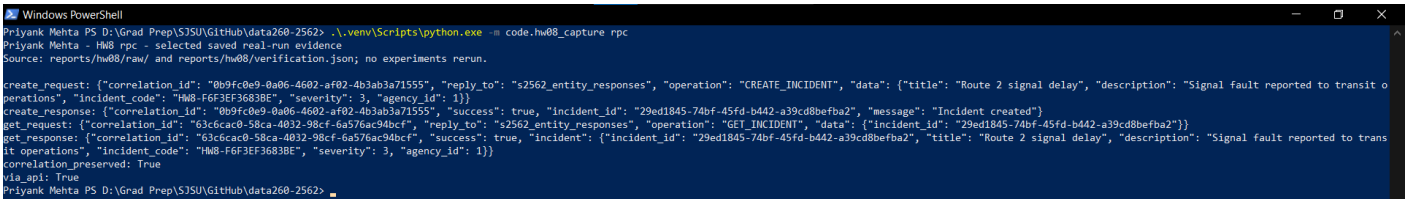


Figure 5. Part 1 successful CREATE_INCIDENT and GET_INCIDENT requests and responses.

Generated incident_id: 29ed1845-74bf-45fd-b442-a39cd8befba2

Part 1 Kafka client

KafkaRPC registers a pending future before publishing a request. The response consumer looks up that future using the unchanged correlation_id. Sending and waiting are bounded by timeouts; the finally block removes the pending entry.

data260-2562 > backend > hw08_rpc.py > KafkaRPC > __init__

```
10  class KafkaRPC:
45      async def receive(self):
46          async for message in self.consumer:
47              try:
48                  data = decode(message.value)
49              except (ValueError, TypeError, UnicodeError) as exc:
50                  emit(self.component, "invalid_response", error=str(exc), topic=message.topic, offset=message.offset)
51                  continue
52              future = self.pending.get(data.get("correlation_id"))
53              if future is not None and not future.done():
54                  emit(
55                      self.component,
56                      "response_received",
57                      topic=message.topic,
58                      partition=message.partition,
59                      offset=message.offset,
60                      message=data,
61                  )
62                  future.set_result(data)
63
64      async def request(self, request, timeout=15):
65          cid = request.get("correlation_id")
66          if not isinstance(cid, str) or not cid.strip():
67              return {"correlation_id": cid, "success": False, "error": "correlation_id is required"}
68          if cid in self.pending:
69              return {"correlation_id": cid, "success": False, "error": "correlation_id is already in flight"}
70          future = asyncio.get_running_loop().create_future()
71          self.pending[cid] = future
72          try:
73              emit(self.component, "request_sent", topic=REQUESTS, message=request)
74              await asyncio.wait_for(self.producer.send_and_wait(REQUESTS, request, key=cid.encode()), timeout=timeout)
75              response = await asyncio.wait_for(future, timeout=timeout)
76              emit(self.component, "client_response", message=response)
77              return response
78          except (TimeoutError, OSError) as exc:
79              return {"correlation_id": cid, "success": False, "error": f"Kafka request failed: {type(exc).__name__}"}
80          finally:
81              self.pending.pop(cid, None)
```

Figure 6. Part 1 request publication, reply matching, timeout handling, and cleanup (backend/hw08_rpc.py).

The CREATE correlation ID is 0b9fc0e9-0a06-4602-af02-4b3ab3a71555. The GET correlation ID is 63c6cac0-58ca-4032-98cf-6a576ac94bcf. These values are created by the client rather than regenerated by the API or worker.

Part 1 Kafka worker

The worker consumes s2562_entity_requests, validates the operation, and invokes its dictionary-backed IncidentService. It sends the result to the validated reply topic, s2562_entity_responses. The offset advances to record.offset + 1 only after response publication succeeds.

data260-2562 > backend > hw08_worker.py > ...

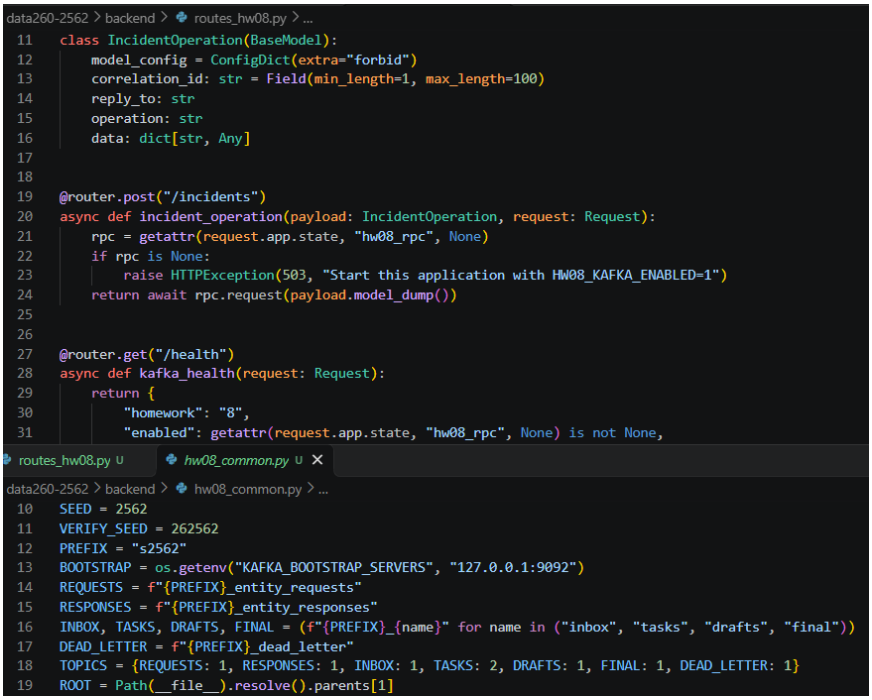
```python
async def run_worker(group="s2562_incident_worker", instance="worker", idempotent=True, start_latest=False):
    try:
        async for record in reader:
            try:
                request = decode(record.value)
            except (ValueError, TypeError, UnicodeError) as exc:
                await sender.send_and_wait(
                    DEAD_LETTER,
                    {
                        "success": False,
                        "error": str(exc),
                        "source_topic": record.topic,
                        "offset": record.offset,
                        "raw": (record.value or b"").decode(errors="replace"),
                    },
                )
            else:
                emit(
                    instance,
                    "request_received",
                    topic=record.topic,
                    partition=record.partition,
                    offset=record.offset,
                    message=request,
                )
                response = service.process(request)
                emit(instance, "request_processed", message=response, incident_count=len(service.incidents))
                # Only the assigned reply topic is allowed: the exercise uses exactly two RPC topics.
                await sender.send_and_wait(RESPONSES, response, key=str(request.get("correlation_id")).encode())
                emit(instance, "response_sent", topic=RESPONSES, message=response)
            await reader.commit({TopicPartition(record.topic, record.partition): record.offset + 1})
            emit(instance, "offset_committed", partition=record.partition, next_offset=record.offset + 1)
    finally:
        await reader.stop()
        await sender.stop()
```

Figure 7. Part 1 worker receive, process, reply, and manual commit sequence (backend/hw08_worker.py).

Part 1 incident storage is intentionally in memory. Restarting the worker clears its incident dictionary and replay cache. Pipeline deduplication, used in Part 3, is stored separately.

Part 1 API wrapper and response schemas

POST /api/hw08/incidents accepts the caller's request envelope and forwards it to rpc.request(). It does not invoke incident business logic directly.



```
data260-2562 > backend > routes_hw08.py > ...
class IncidentOperation(BaseModel):
    model_config = ConfigDict(extra="forbid")
    correlation_id: str = Field(min_length=1, max_length=100)
    reply_to: str
    operation: str
    data: dict[str, Any]


@router.post("/incidents")
async def incident_operation(payload: IncidentOperation, request: Request):
    rpc = getattr(request.app.state, "hw08_rpc", None)
    if rpc is None:
        raise HTTPException(503, "Start this application with HW08_KAFKA_ENABLED=1")
    return await rpc.request(payload.model_dump())


@router.get("/health")
async def kafka_health(request: Request):
    return {
        "homework": "8",
        "enabled": getattr(request.app.state, "hw08_rpc", None) is not None,
```

routes_hw08.py U | hw08_common.py U

```
data260-2562 > backend > hw08_common.py > ...
SEED = 2562
VERIFY_SEED = 262562
PREFIX = "s2562"
BOOTSTRAP = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "127.0.0.1:9092")
REQUESTS = f"{PREFIX}_entity_requests"
RESPONSES = f"{PREFIX}_entity_responses"
INBOX, TASKS, DRAFTS, FINAL = (f"{PREFIX}_{name}" for name in ("inbox", "tasks", "drafts", "final"))
DEAD_LETTER = f"{PREFIX}_dead_letter"
TOPICS = {REQUESTS: 1, RESPONSES: 1, INBOX: 1, TASKS: 2, DRAFTS: 1, FINAL: 1, DEAD_LETTER: 1}
ROOT = Path(__file__).resolve().parents[1]
```

Figure 8. Parts 1-2 API request model, Kafka forwarding, and prefixed topic definitions.

Response variant	Required fields
CREATE success	correlation_id: string; success: true; incident_id: UUID string; message: "Incident created"
GET success	correlation_id: string; success: true; incident: {incident_id, title, description, incident_code, severity, agency_id}
Failure	correlation_id: original request ID; success: false; error: descriptive string

Observed error results

Case	Observed response
Unknown UUID	success=false; error="Incident not found"
Missing fields	success=false; error identifies all five required incident fields
Unknown operation	success=false; error="Unknown operation"

Part 1 Complete correlation flow

The following excerpt records CREATE followed by GET in timestamp order. For each operation, the correlation ID remains unchanged from API publication through worker processing and the returned client response. Figure 9 retains both full IDs and both topic names.

Operation	Time UTC	Observed event
CREATE	14:48:25.645	api.request_sent
CREATE	14:48:25.655	worker.request_received
CREATE	14:48:25.661	worker.request_processed
CREATE	14:48:25.669	worker.response_sent
CREATE	14:48:25.670	api.response_received
CREATE	14:48:25.675	api.client_response
GET	14:48:25.688	api.request_sent
GET	14:48:25.695	worker.request_received
GET	14:48:25.702	worker.request_processed
GET	14:48:25.710	worker.response_sent
GET	14:48:25.711	api.response_received
GET	14:48:25.717	api.client_response

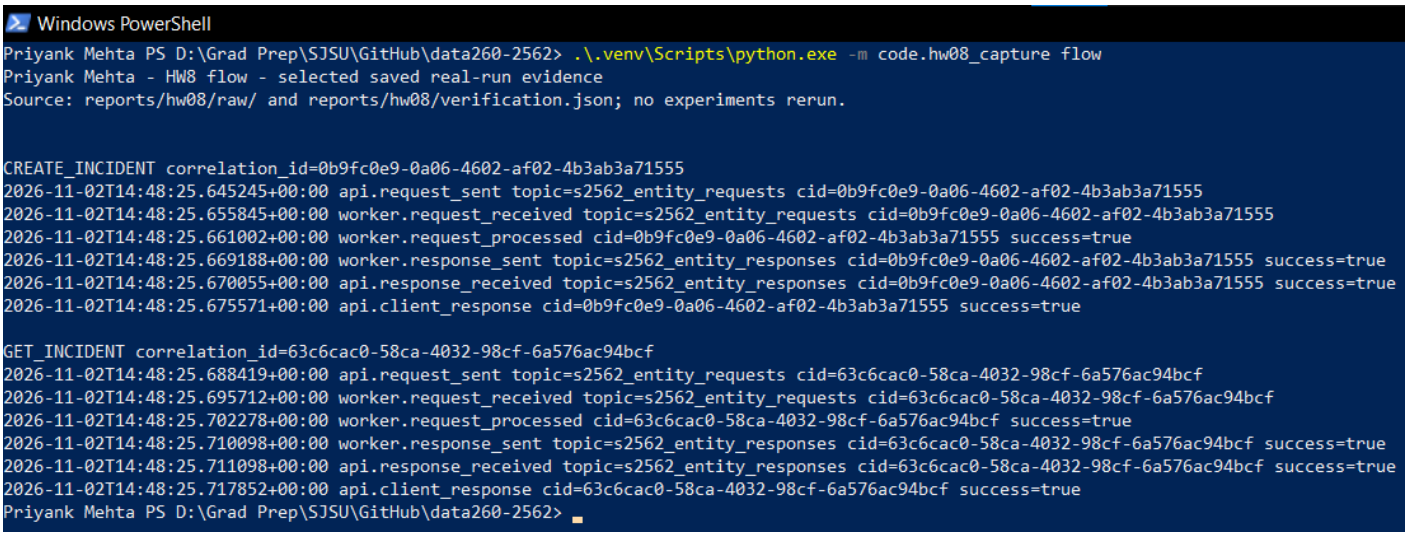


Figure 9. Part 1 complete CREATE then GET send, receive, process, reply, and client-response logs.

Retrieved incident: Route 2 signal delay; incident code HW8-F6F3EF3683BE; severity 3; agency_id 1. The incident_id matches the UUID generated by CREATE.

Part 2 Three agent Kafka pipeline

Each role consumes its incoming topic and publishes a message to the next topic. Planner creates a transit-response plan, Writer produces a draft, and Reviewer checks for a nonempty draft and returns status="approved" with an answer. No agent invokes another agent directly.

Process	Reads	Publishes
Sender	-	s2562_inbox
Planner	s2562_inbox	s2562_tasks
Writer	s2562_tasks	s2562_drafts
Reviewer	s2562_drafts	s2562_final



Figure 10. Part 2 stage routing and Planner, Writer, and Reviewer logic.



Figure 11. Part 2 separate process entry points: planner.py, writer.py, and reviewer.py.

Part 2 Consumption publication and commits

The consumer logs its topic, group, partition, and offset. The default path processes the message, publishes its result, and then commits the next offset. The unsafe early-commit branch is enabled only for the controlled before-fix experiment.



Figure 12. Part 2 incoming Kafka consumption and the bounded attempt loop.

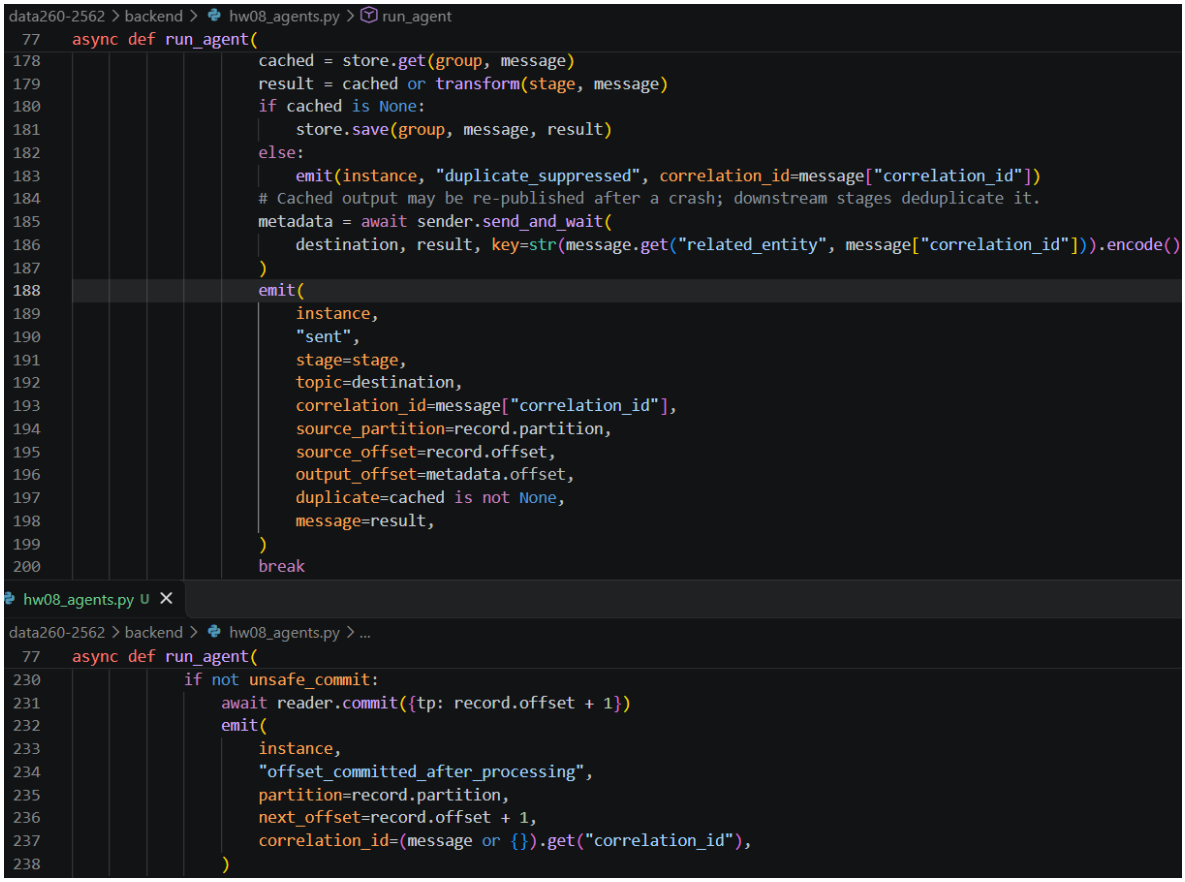


Figure 13. Parts 2-3 cached-result handling, output publication, and commit after processing.

Part 2 Complete message journey

Question: How should transit staff handle a Route 2 signal delay?

Correlation ID: 98db059e-7159-4ecb-9dc5-8adf5f97a4ba

Time UTC	Stage and topic
14:48:28.907	question -> s2562_inbox
14:48:28.963	planner -> s2562_tasks
14:48:29.607	writer-1 -> s2562_drafts
14:48:29.658	reviewer -> s2562_final

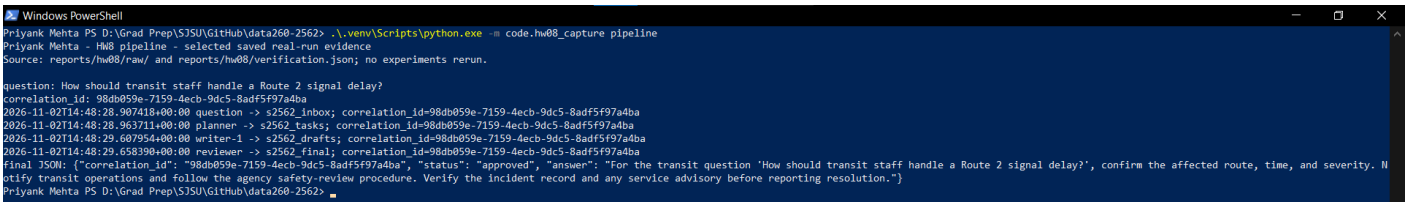


Figure 14. Part 2 inbox-to-final journey and the approved final JSON response.

Final result

Field	Recorded value
correlation_id	98db059e-7159-4ecb-9dc5-8adf5f97a4ba
status	approved
answer	For the transit question 'How should transit staff handle a Route 2 signal delay?', confirm the affected route, time, and severity. Notify transit operations and follow the agency safety-review procedure. Verify the incident record and any service advisory before reporting resolution.

Kafka coordinates the hand-offs through retained topic records and consumer-group offsets. Writer can stop and restart without a direct call from Planner failing, while Reviewer independently consumes completed drafts. The same correlation ID connects the records from all four topics.

Part 3 Failure behavior

The before/after experiments used actual Kafka records and controlled consumer processes. Baseline flags reproduce duplicate side effects, early-commit loss, and poison-message blocking. The corrected path enables replay handling, commits after work, and sends exhausted poison inputs to s2562_dead_letter.

Failure scenario	Before fix	After fix
Duplicate delivery	Replaying one CREATE correlation produced 2 incidents with different UUIDs.	Replaying one CREATE correlation returned the same UUID; incident count remained 1.
Consumer killed mid-work	Source offset 0 was already committed as 1 when Writer was killed; the interrupted record was skipped after restart.	Source offset 2 remained committed at 2 when killed. Restart processed offset 2 and then committed 3.
Poison message	Invalid input at offset 3 blocked the partition without committing past it.	Three attempts, dead-letter publication, then successful processing of the following input at offset 4.
Two partitions and two consumers	20 unkeyed related inputs used partitions 0 and 1; completion order differed from input order.	Key route-2 placed all 20 related inputs in partition 1; completion order was 0 through 19.

Evidence files

raw/duplicate_delivery.json; raw/consumer_death.json; raw/poison_message.json; raw/ordering_unkeyed.json and .csv; raw/ordering_keyed.json and .csv. RUN_LOG.txt contains the full timestamped command output and event traces.

Part 3 I Duplicate delivery

IncidentService caches responses by correlation_id and the operation/data fingerprint. Replaying the same input returns the cached UUID without inserting another incident. Reusing an ID for different input returns an error. Pipeline stages use a separate SQLite cache keyed by consumer scope and correlation_id, with a fingerprint check and stored output.

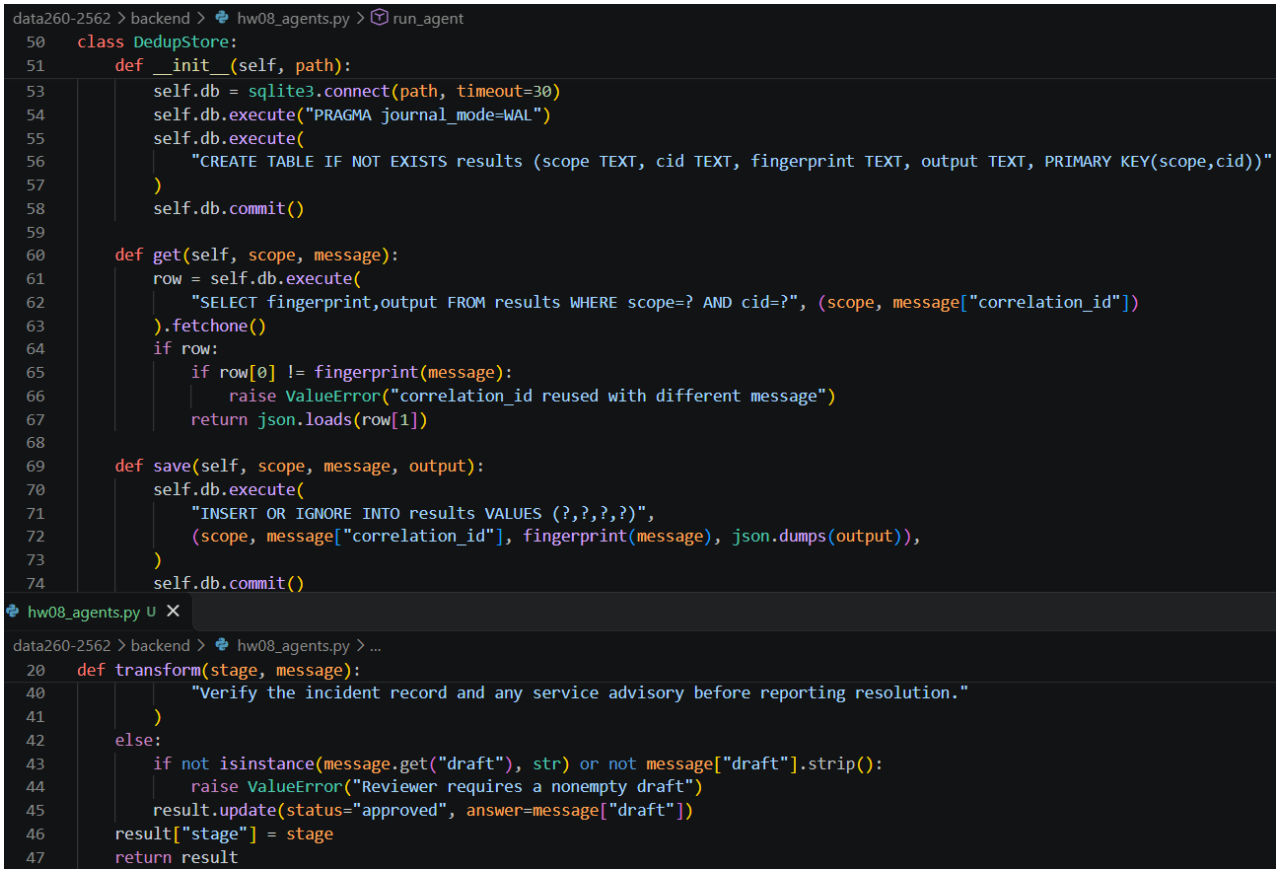


Figure 15. Part 3.I persistent pipeline deduplication (DedupStore in backend/hw08_agents.py).

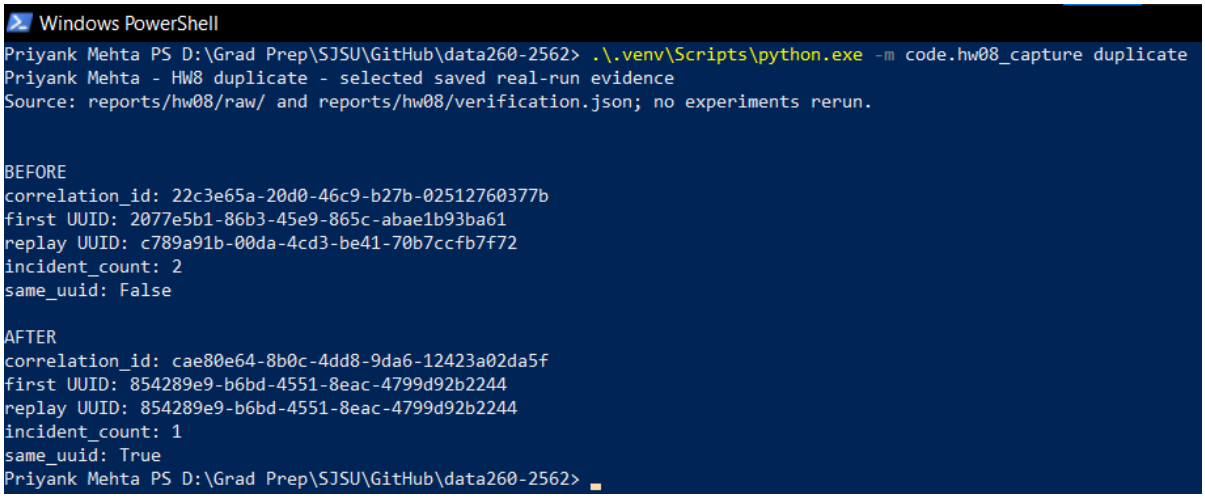
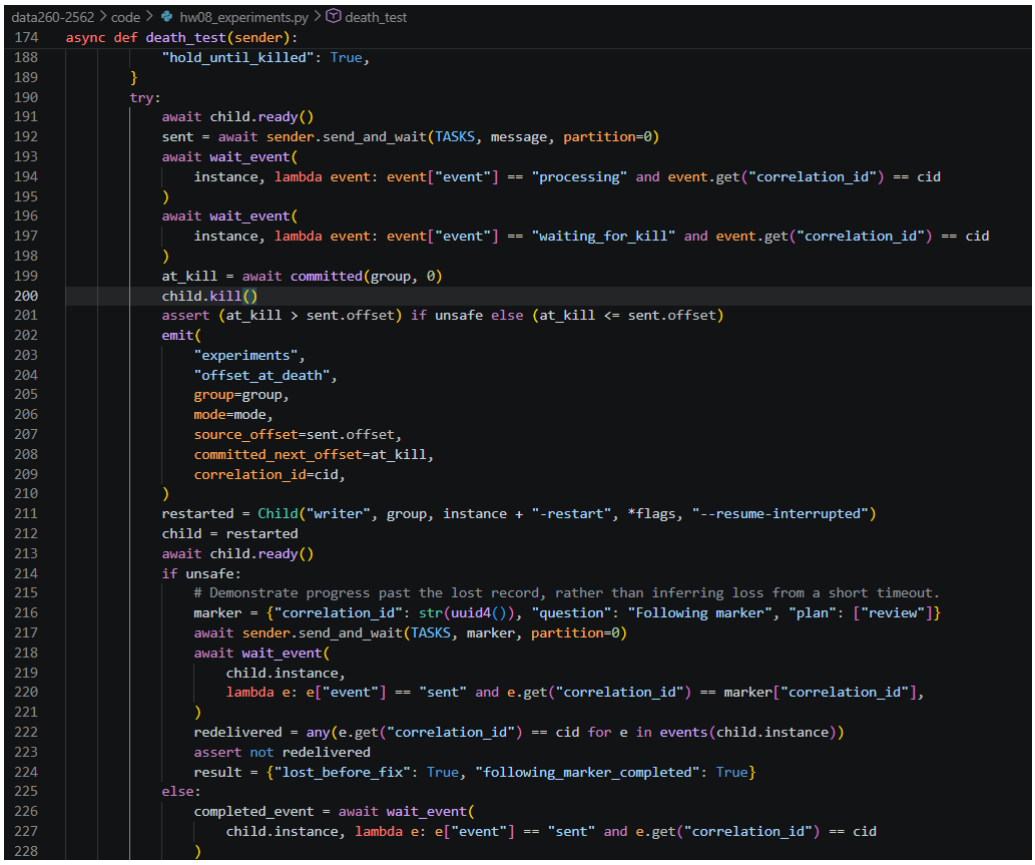


Figure 16. Part 3.I replay counts and UUIDs before and after incident-service idempotency.

Incident replay handling lasts for the worker lifetime. Pipeline caching survives Writer restart. Output publication is at least once: a cached output can be republished after a crash, so downstream stages also apply deduplication.

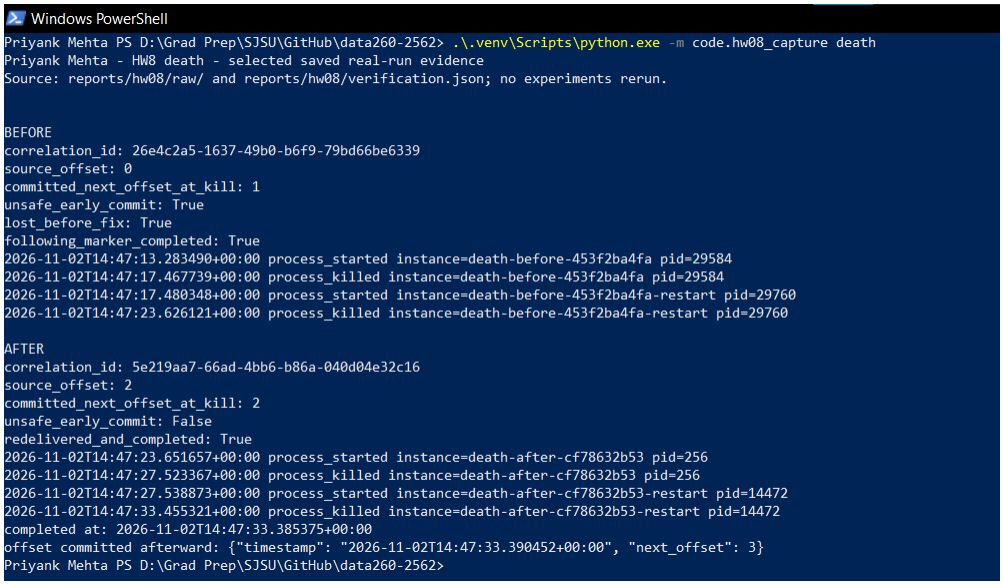
Part 3 II Consumer death

The experiment waits until Writer is inside processing, records its committed offset, terminates that child process, and restarts it in the same group. Early commit loses the interrupted record; committing after publication permits redelivery. The successful retry publishes the draft before committing the next offset.



```
data260-2562 > code > hw08_experiments.py > death_test
async def death_test(sender):
            "hold_until_killed": True,
        }
        try:
            await child.ready()
            sent = await sender.send_and_wait(TASKS, message, partition=0)
            await wait_event(
                instance, lambda event: event["event"] == "processing" and event.get("correlation_id") == cid
            )
            await wait_event(
                instance, lambda event: event["event"] == "waiting_for_kill" and event.get("correlation_id") == cid
            )
            at_kill = await committed(group, 0)
            child.kill()
            assert (at_kill > sent.offset) if unsafe else (at_kill <= sent.offset)
            emit(
                "experiments",
                "offset_at_death",
                group=group,
                mode=mode,
                source_offset=sent.offset,
                committed_next_offset=at_kill,
                correlation_id=cid,
            )
            restarted = Child("writer", group, instance + "-restart", *flags, "--resume-interrupted")
            child = restarted
            await child.ready()
            if unsafe:
                # Demonstrate progress past the lost record, rather than inferring loss from a short timeout.
                marker = {"correlation_id": str(uuid4()), "question": "Following marker", "plan": ["review"]}
                await sender.send_and_wait(TASKS, marker, partition=0)
                await wait_event(
                    child.instance,
                    lambda e: e["event"] == "sent" and e.get("correlation_id") == marker["correlation_id"],
                )
                redelivered = any(e.get("correlation_id") == cid for e in events(child.instance))
                assert not redelivered
                result = {"lost_before_fix": True, "following_marker_completed": True}
            else:
                completed_event = await wait_event(
                    child.instance, lambda e: e["event"] == "sent" and e.get("correlation_id") == cid
                )
```

Figure 17. Part 3.II controlled Writer termination, restart, and offset assertions.



```
Windows PowerShell
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562> .\.venv\Scripts\python.exe -m code.hw08_capture death
Priyank Mehta - HW8 death - selected saved real-run evidence
Source: reports/hw08/raw/ and reports/hw08/verification.json; no experiments rerun.


BEFORE
correlation_id: 26e4c2a5-1637-49b0-b6f9-79bd66be6339
source_offset: 0
committed_next_offset_at_kill: 1
unsafe_early_commit: True
lost_before_fix: True
following_marker_completed: True
2026-11-02T14:47:13.283490+00:00 process_started instance=death-before-453f2ba4fa pid=29584
2026-11-02T14:47:17.467739+00:00 process_killed instance=death-before-453f2ba4fa pid=29584
2026-11-02T14:47:17.480348+00:00 process_started instance=death-before-453f2ba4fa-restart pid=29760
2026-11-02T14:47:23.626121+00:00 process_killed instance=death-before-453f2ba4fa-restart pid=29760

AFTER
correlation_id: 5e219aa7-66ad-4bb6-b86a-040d04e32c16
source_offset: 2
committed_next_offset_at_kill: 2
unsafe_early_commit: False
redelivered_and_completed: True
2026-11-02T14:47:23.651657+00:00 process_started instance=death-after-cf78632b53 pid=256
2026-11-02T14:47:27.523367+00:00 process_killed instance=death-after-cf78632b53 pid=256
2026-11-02T14:47:27.538873+00:00 process_started instance=death-after-cf78632b53-restart pid=14472
2026-11-02T14:47:33.455321+00:00 process_killed instance=death-after-cf78632b53-restart pid=14472
completed at: 2026-11-02T14:47:33.385375+00:00
offset committed afterward: {"timestamp": "2026-11-02T14:47:33.390452+00:00", "next_offset": 3}
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

Figure 18. Part 3.II process lifecycle and before/after offset evidence.

Part 3 III Poison message

The poison input contains a numeric question instead of nonempty text. The baseline leaves it uncommitted and blocks subsequent work in that partition. The corrected loop makes three total attempts, waits 0.1 and 0.2 seconds between attempts, publishes the failure envelope to s2562_dead_letter, and then commits. The valid message at offset 4 proceeds after the bad record at offset 3.

data260-2562 > backend > hw08_agents.py > run_agent

```
77   async def run_agent(
201                 except (ValueError, TypeError, KeyError, UnicodeError) as exc:
202                     emit(
203                         instance,
204                         "retry",
205                         attempt=attempt,
206                         error=str(exc),
207                         correlation_id=(message or {}).get("correlation_id"),
208                         partition=record.partition,
209                         offset=record.offset,
210                     )
211                     if halt_on_poison:
212                         # Explicit before-fix mode: leave the offset uncommitted and later work blocked.
213                         emit(instance, "poison_blocks_partition", partition=record.partition, offset=record.offset)
214                         await asyncio.Event().wait()
215                     if attempt == 3:
216                         dead = {
217                             "correlation_id": (message or {}).get("correlation_id"),
218                             "success": False,
219                             "error": str(exc),
220                             "attempts": attempt,
221                             "source_topic": source,
222                             "partition": record.partition,
223                             "offset": record.offset,
224                             "original": (record.value or b"").decode(errors="replace"),
225                         }
226                         await sender.send_and_wait(DEAD_LETTER, dead)
227                         emit(instance, "dead_letter", message=dead, topic=DEAD_LETTER)
228                     else:
229                         await asyncio.sleep(0.1 * 2 ** (attempt - 1))
230             if not unsafe_commit:
231                 await reader.commit({tp: record.offset + 1})
232                 emit(
233                     instance,
234                     "offset_committed_after_processing",
235                     partition=record.partition,
236                     next_offset=record.offset + 1,
237                     correlation_id=(message or {}).get("correlation_id"),
238                 )
```

Figure 19. Part 3.III bounded retries, exponential backoff, dead-letter publication, and manual commit.

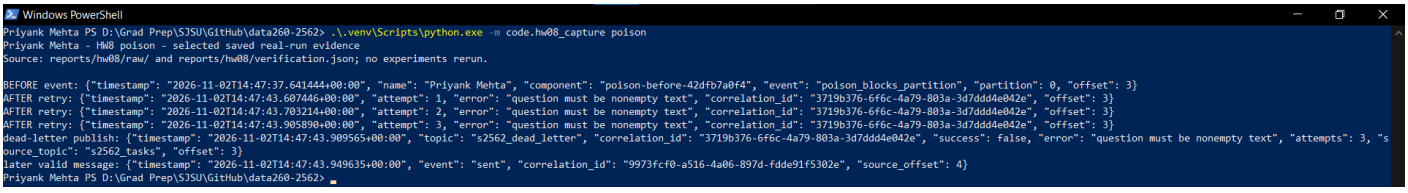


Figure 20. Part 3.III three failed attempts, the dead-letter result, and later successful work.

Part 3 IV Ordering experiment

Both runs generate 20 related route-2 messages using VERIFY_SEED=262562. The tasks topic has two partitions, and two Writer instances join the same consumer group. The unkeyed baseline explicitly alternates partitions. The second run uses the related entity as the Kafka key; all records for route-2 then share one partition.

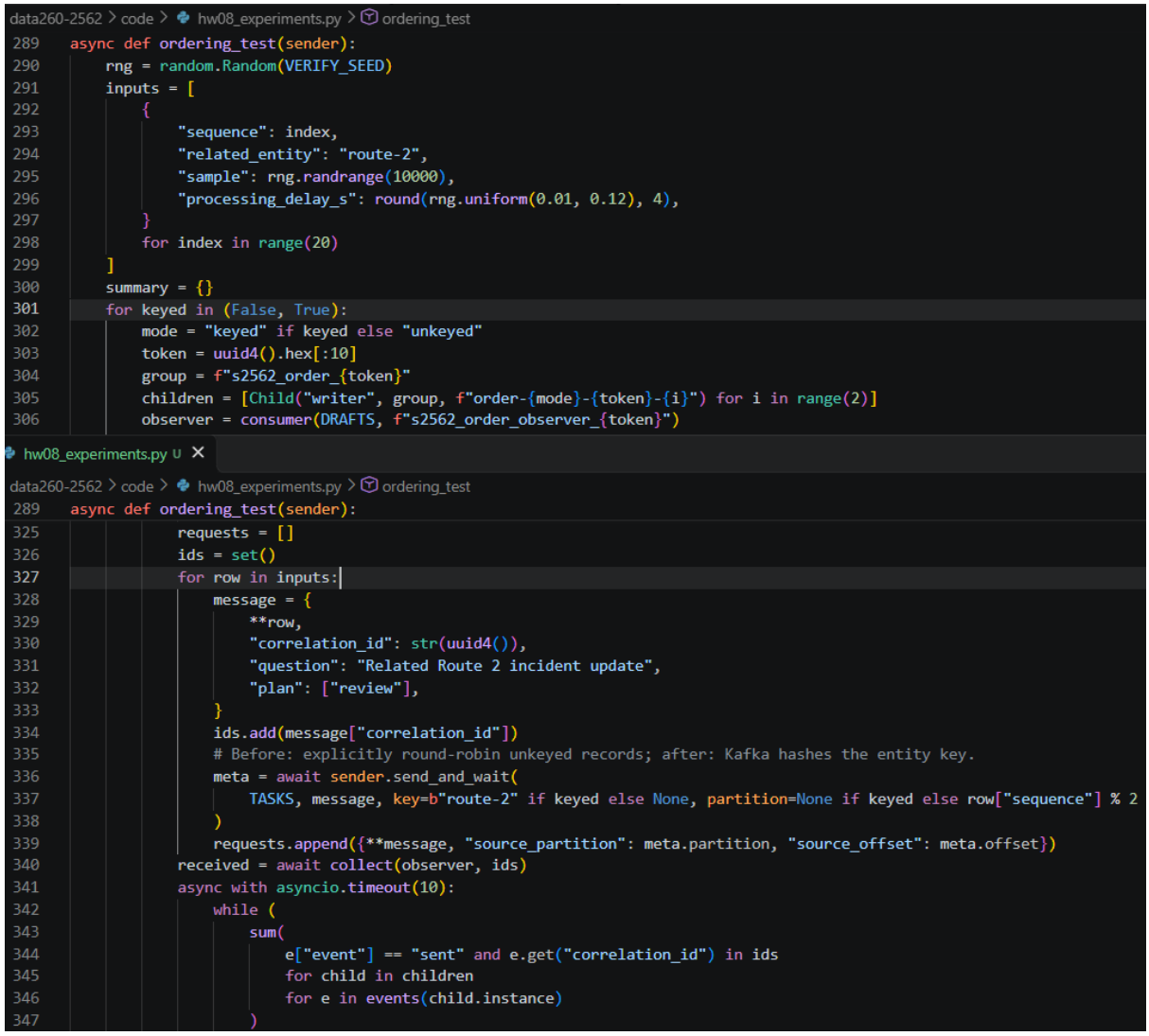
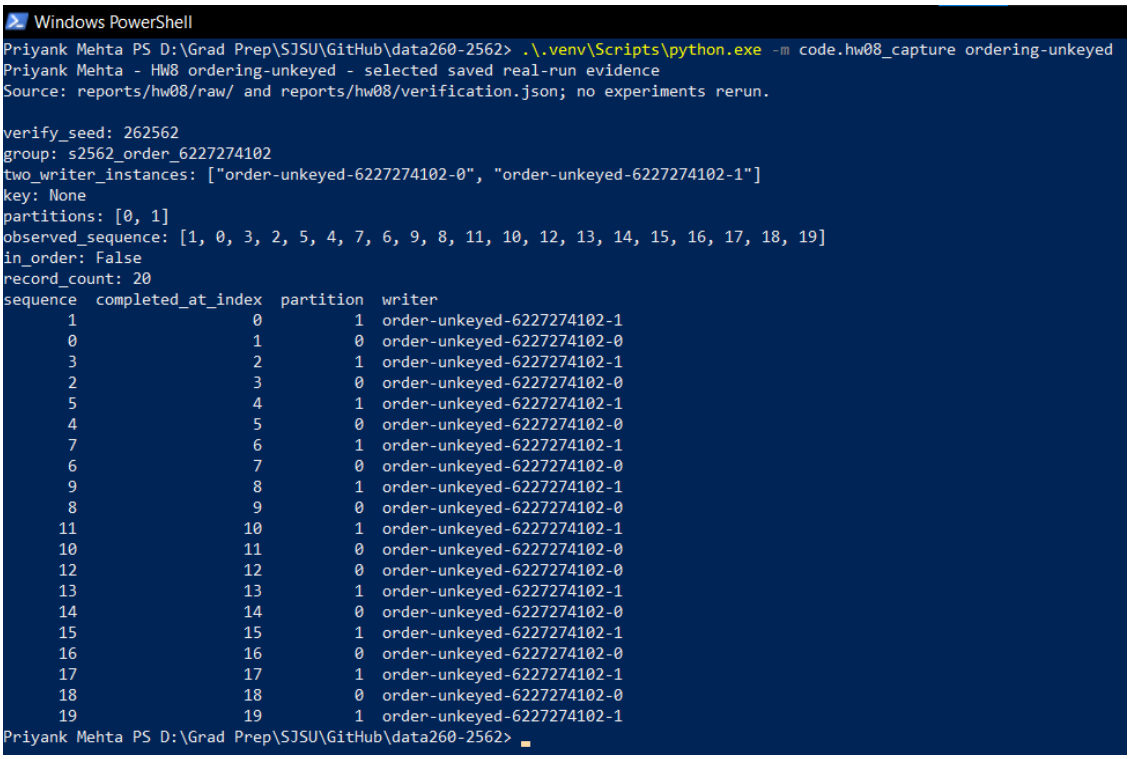


Figure 21. Part 3.IV seeded input generation, two Writer instances, and partition-key selection.

Run	Messages	Partitions	Input order preserved
Unkeyed	20	0 and 1	No
Entity key route-2	20	1	Yes

Part 3 IV Observed completion order

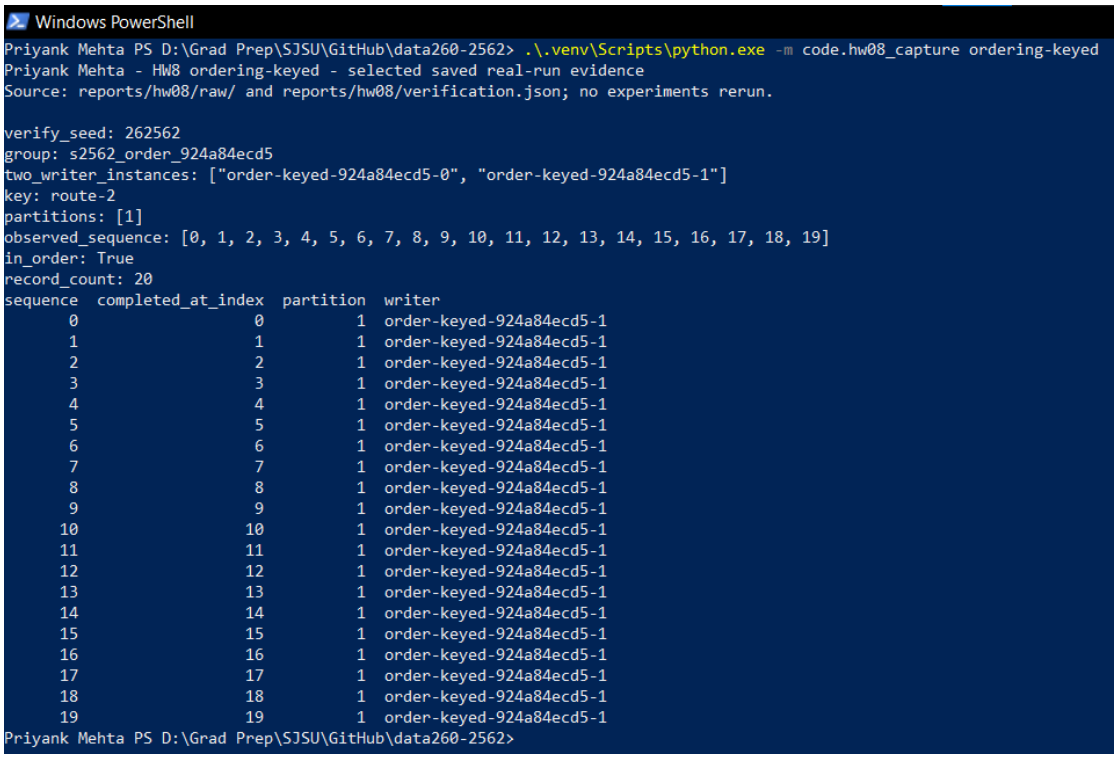
Unkeyed records were handled by both Writers and completed out of global order. Entity-keyed records went to one Writer within the two-member group, preserving all 20 sequence positions. Kafka ordering is per partition; it does not imply a global order across partitions.



```
Windows PowerShell
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562> .\.venv\Scripts\python.exe -m code.hw08_capture ordering-unkeyed
Priyank Mehta - HW8 ordering-unkeyed - selected saved real-run evidence
Source: reports/hw08/raw/ and reports/hw08/verification.json; no experiments rerun.

verify_seed: 262562
group: s2562_order_6227274102
two_writer_instances: ["order-unkeyed-6227274102-0", "order-unkeyed-6227274102-1"]
key: None
partitions: [0, 1]
observed_sequence: [1, 0, 3, 2, 5, 4, 7, 6, 9, 8, 11, 10, 12, 13, 14, 15, 16, 17, 18, 19]
in_order: False
record_count: 20
sequence  completed_at_index  partition  writer
       1                   0          1  order-unkeyed-6227274102-1
       0                   1          0  order-unkeyed-6227274102-0
       3                   2          1  order-unkeyed-6227274102-1
       2                   3          0  order-unkeyed-6227274102-0
       5                   4          1  order-unkeyed-6227274102-1
       4                   5          0  order-unkeyed-6227274102-0
       7                   6          1  order-unkeyed-6227274102-1
       6                   7          0  order-unkeyed-6227274102-0
       9                   8          1  order-unkeyed-6227274102-1
       8                   9          0  order-unkeyed-6227274102-0
      11                  10          1  order-unkeyed-6227274102-1
      10                  11          0  order-unkeyed-6227274102-0
      12                  12          0  order-unkeyed-6227274102-0
      13                  13          1  order-unkeyed-6227274102-1
      14                  14          0  order-unkeyed-6227274102-0
      15                  15          1  order-unkeyed-6227274102-1
      16                  16          0  order-unkeyed-6227274102-0
      17                  17          1  order-unkeyed-6227274102-1
      18                  18          0  order-unkeyed-6227274102-0
      19                  19          1  order-unkeyed-6227274102-1
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

Figure 22. Part 3.IV unkeyed results: two partitions, two active Writers, and out-of-order completion.



```
Windows PowerShell
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562> .\.venv\Scripts\python.exe -m code.hw08_capture ordering-keyed
Priyank Mehta - HW8 ordering-keyed - selected saved real-run evidence
Source: reports/hw08/raw/ and reports/hw08/verification.json; no experiments rerun.

verify_seed: 262562
group: s2562_order_924a84ecd5
two_writer_instances: ["order-keyed-924a84ecd5-0", "order-keyed-924a84ecd5-1"]
key: route-2
partitions: [1]
observed_sequence: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
in_order: True
record_count: 20
sequence  completed_at_index  partition  writer
       0                   0          1  order-keyed-924a84ecd5-1
       1                   1          1  order-keyed-924a84ecd5-1
       2                   2          1  order-keyed-924a84ecd5-1
       3                   3          1  order-keyed-924a84ecd5-1
       4                   4          1  order-keyed-924a84ecd5-1
       5                   5          1  order-keyed-924a84ecd5-1
       6                   6          1  order-keyed-924a84ecd5-1
       7                   7          1  order-keyed-924a84ecd5-1
       8                   8          1  order-keyed-924a84ecd5-1
       9                   9          1  order-keyed-924a84ecd5-1
      10                  10          1  order-keyed-924a84ecd5-1
      11                  11          1  order-keyed-924a84ecd5-1
      12                  12          1  order-keyed-924a84ecd5-1
      13                  13          1  order-keyed-924a84ecd5-1
      14                  14          1  order-keyed-924a84ecd5-1
      15                  15          1  order-keyed-924a84ecd5-1
      16                  16          1  order-keyed-924a84ecd5-1
      17                  17          1  order-keyed-924a84ecd5-1
      18                  18          1  order-keyed-924a84ecd5-1
      19                  19          1  order-keyed-924a84ecd5-1
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

Figure 23. Part 3.IV entity-keyed results: one partition and sequence 0 through 19.

Part 3 V Comparison with HW2

The HW2 graph executed function calls within one process, with each step waiting directly for the previous step. Kafka moves those hand-offs into broker records and consumer offsets.

Category	Change	Effect compared with HW2
Benefit 1	Independent scaling	Add Writer consumers to a group to distribute task partitions without running Planner or Reviewer in the same process.
Benefit 2	Decoupled failure	A stopped consumer can restart and resume from its committed offset while Kafka retains its input.
Responsibility 1	Duplicates and acknowledgments	Detect replays and commit offsets after successful processing so retries do not add side effects or lose interrupted work.
Responsibility 2	Ordering and poison inputs	Choose keys for related records and bound retries with a dead-letter route so an invalid input cannot block later work.

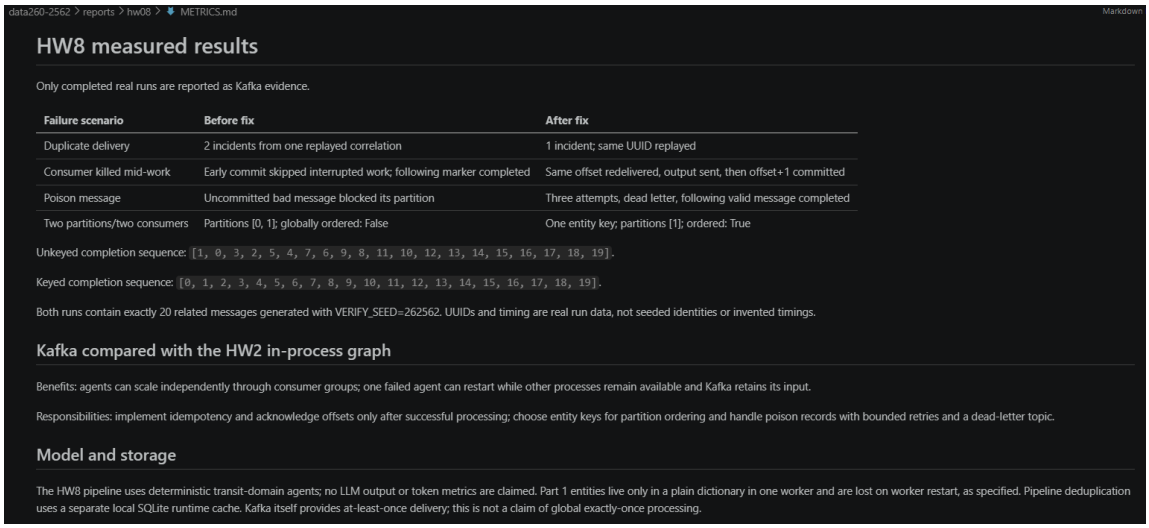
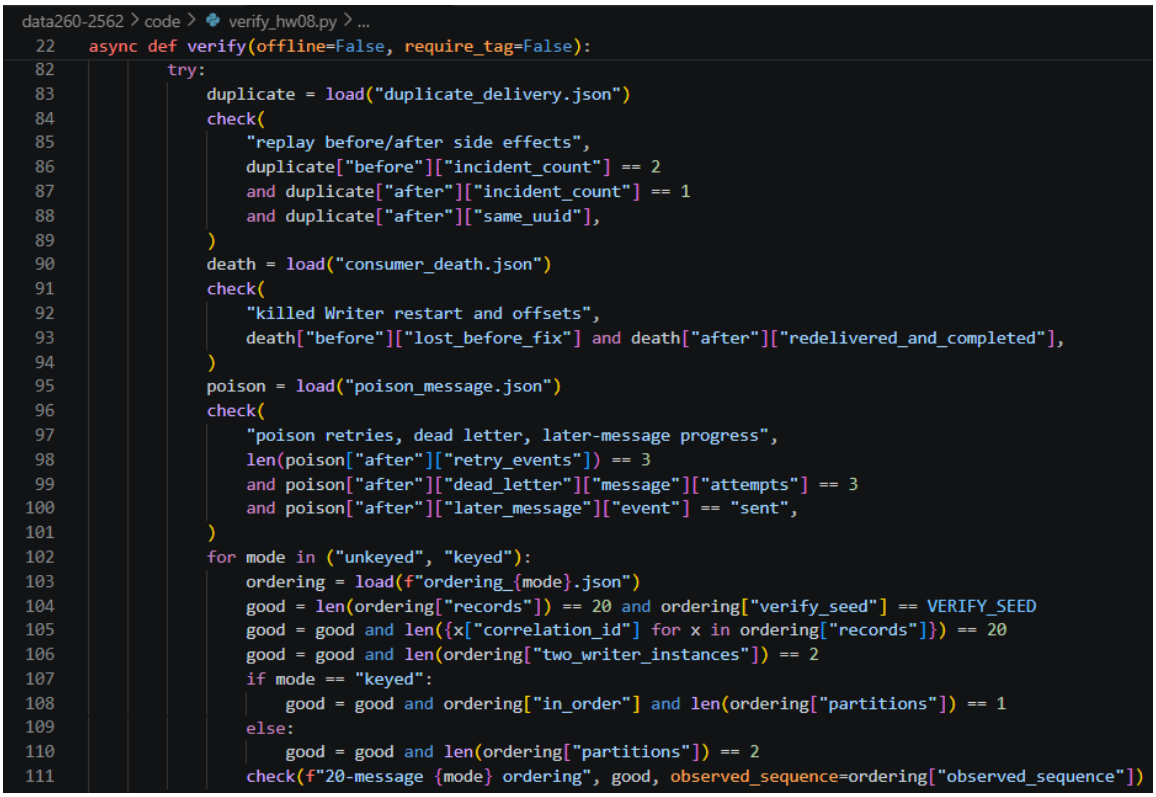


Figure 24. Part 3.V recorded failure metrics and the HW2 comparison.

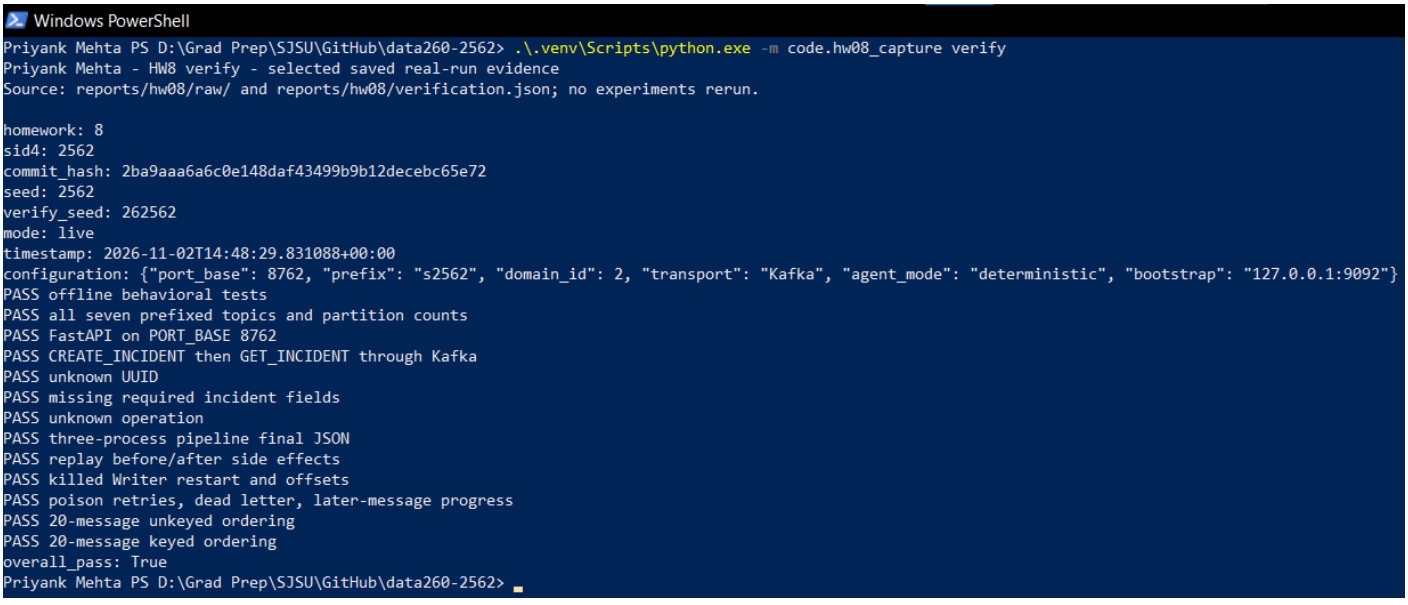
Verification

verify_hw08.py performs objective smoke checks on API behavior, topic partition counts, the three-process pipeline, and the saved failure records. The live run at 2026-11-02 14:48:29 UTC passed 13 of 13 checks; its offline behavioral suite passed 14 of 14 tests.



```
data260-2562 > code > verify_hw08.py > ...
 22   async def verify(offline=False, require_tag=False):
 82           try:
 83               duplicate = load("duplicate_delivery.json")
 84               check(
 85                   "replay before/after side effects",
 86                   duplicate["before"]["incident_count"] == 2
 87                   and duplicate["after"]["incident_count"] == 1
 88                   and duplicate["after"]["same_uuid"],
 89               )
 90               death = load("consumer_death.json")
 91               check(
 92                   "killed Writer restart and offsets",
 93                   death["before"]["lost_before_fix"] and death["after"]["redelivered_and_completed"],
 94               )
 95               poison = load("poison_message.json")
 96               check(
 97                   "poison retries, dead letter, later-message progress",
 98                   len(poison["after"]["retry_events"]) == 3
 99                   and poison["after"]["dead_letter"]["message"]["attempts"] == 3
100                   and poison["after"]["later_message"]["event"] == "sent",
101               )
102               for mode in ("unkeyed", "keyed"):
103                   ordering = load(f"ordering_{mode}.json")
104                   good = len(ordering["records"]) == 20 and ordering["verify_seed"] == VERIFY_SEED
105                   good = good and len({x["correlation_id"] for x in ordering["records"]}) == 20
106                   good = good and len(ordering["two_writer_instances"]) == 2
107                   if mode == "keyed":
108                       good = good and ordering["in_order"] and len(ordering["partitions"]) == 1
109                   else:
110                       good = good and len(ordering["partitions"]) == 2
111                   check(f"20-message {mode} ordering", good, observed_sequence=ordering["observed_sequence"])
```

Figure 25. Objective duplicate, restart, poison, and 20-record ordering checks (code/verify_hw08.py).



```
Windows PowerShell
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562> .\.venv\Scripts\python.exe -m code.hw08_capture verify
Priyank Mehta - HW8 verify - selected saved real-run evidence
Source: reports/hw08/raw/ and reports/hw08/verification.json; no experiments rerun.

homework: 8
sid4: 2562
commit_hash: 2ba9aaa6a6c0e148daf43499b9b12decebc65e72
seed: 2562
verify_seed: 262562
mode: live
timestamp: 2026-11-02T14:48:29.831088+00:00
configuration: {"port_base": 8762, "prefix": "s2562", "domain_id": 2, "transport": "Kafka", "agent_mode": "deterministic", "bootstrap": "127.0.0.1:9092"}
PASS offline behavioral tests
PASS all seven prefixed topics and partition counts
PASS FastAPI on PORT_BASE 8762
PASS CREATE_INCIDENT then GET_INCIDENT through Kafka
PASS unknown UUID
PASS missing required incident fields
PASS unknown operation
PASS three-process pipeline final JSON
PASS replay before/after side effects
PASS killed Writer restart and offsets
PASS poison retries, dead letter, later-message progress
PASS 20-message unkeyed ordering
PASS 20-message keyed ordering
overall_pass: True
Priyank Mehta PS D:\Grad Prep\SJSU\GitHub\data260-2562>
```

Figure 26. Live verification metadata, passing checks, and overall_pass=true.

AI Use

Required repository evidence

reports/hw08/ contains RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json, and report.pdf. The verification entry point is verify_hw08.py; the service demonstration is homework_demo.py.

1. What did I use an AI assistant for, and what did I do myself?

I used ChatGPT to troubleshoot installation and command-line issues.

I independently implemented and tested the code and scripts.

2. What AI-produced output was wrong or unsuitable?

It suggested an incorrect command for a python dependency installation.

3. How did I detect the problem or verify the result?

The command resulted in error on execution.

4. What did I change, and why does it work now?

I checked the command through the documentation and executed it, which installed the python dependency successfully.